5 - Sécuriser et tester une API avec Node.js et Mocha

Astuces :

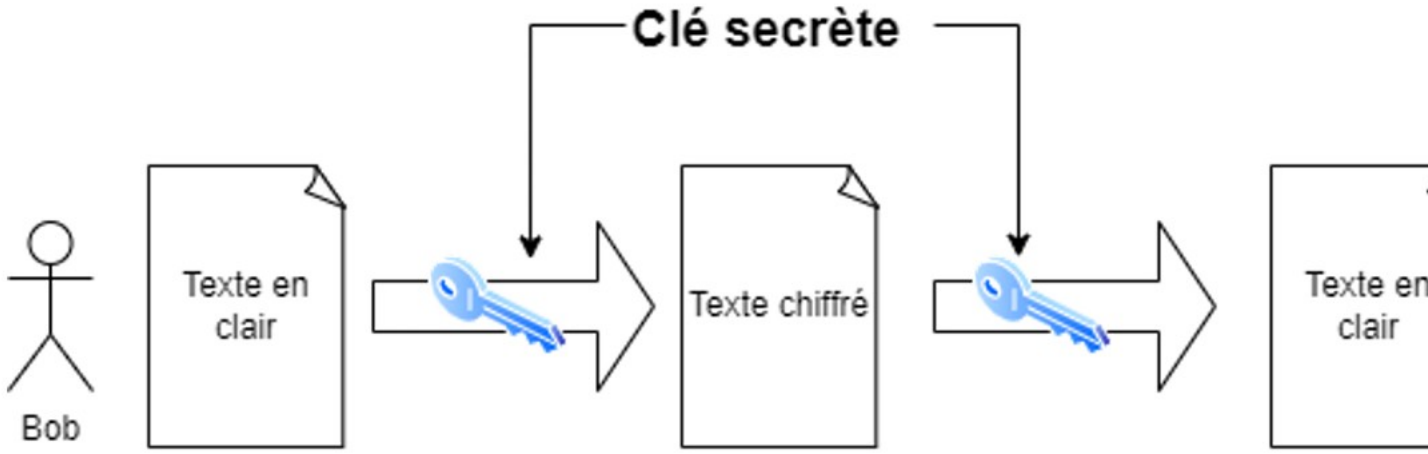
Découvrez le mécanisme de chiffrement

Le chiffrement se base sur des protocoles d'échanges de clé. Ceux-ci permettent à deux utilisateurs d'effectuer des tâches d'authentification mutuelle. Il en existe deux types :

- Les protocoles qui ne nécessitent aucune connaissance préalable d'informations entre les deux utilisateurs (**Diffie-Hellman**), ou chiffrement symétrique.
- Les protocoles qui nécessitent le partage préalable d'une information (clé publique) entre les des deux utilisateurs tels que **RSA utilisé par HTTPS**. Ce sont les chiffrements asymétriques ou hybrides.

Le chiffrement symétrique

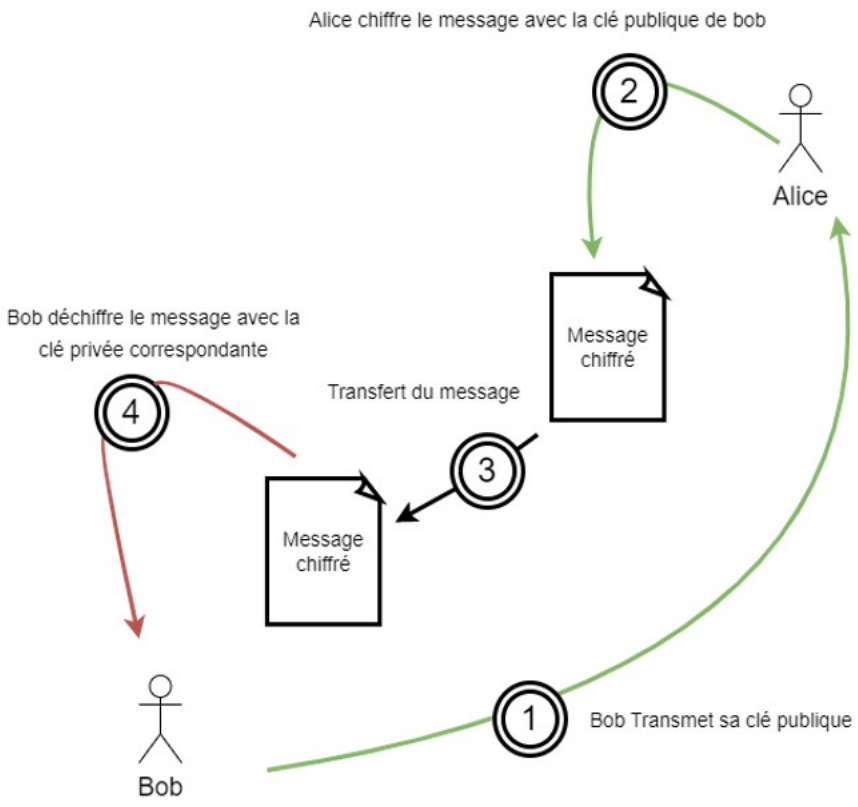
Le chiffrement symétrique utilise une seule clé pour chiffrer et déchiffrer les données. L'émetteur partage cette clé avec le destinataire. Lorsque le destinataire reçoit le message, il devra entrer la clé secrète pour le déchiffrer.



- **Avantage** : Sa facilité de mise en place et son fonctionnement simple à comprendre.
- **Inconvénient** : La clé secrète doit être partagée avec le destinataire à un moment ou à un autre.

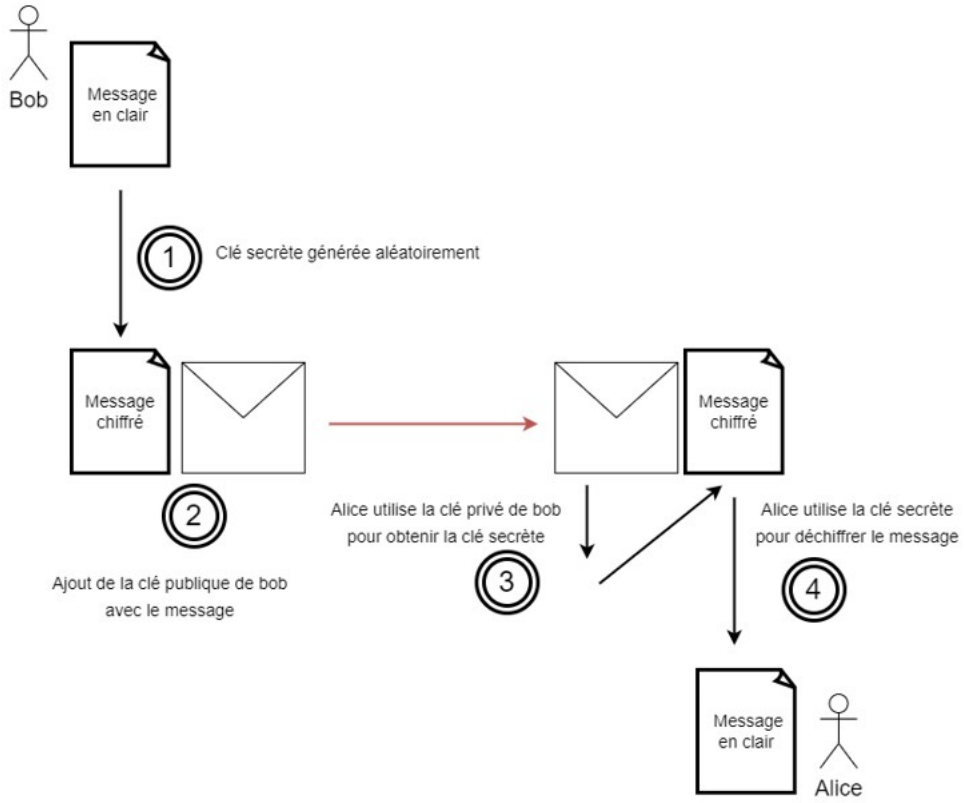
Le chiffrement asymétrique

Le chiffrement asymétrique, plus complexe à comprendre, (**Public-key cryptography**) est utilisé pour protéger des systèmes de fichiers contre les accès non autorisés et pour échanger des messages secrets.



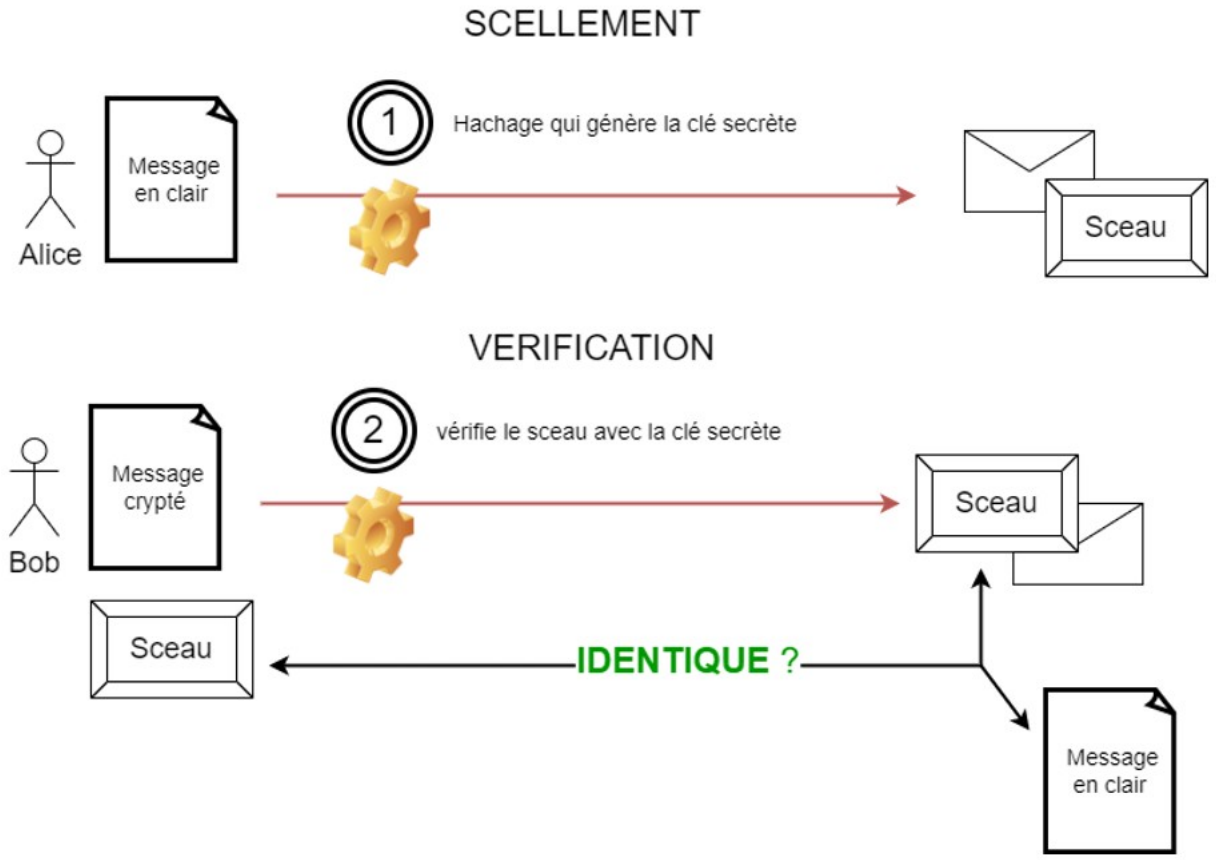
Le chiffrement hybride

C’est un compromis entre le chiffrement symétrique et asymétrique.



Mettez en place un scellement (MAC)

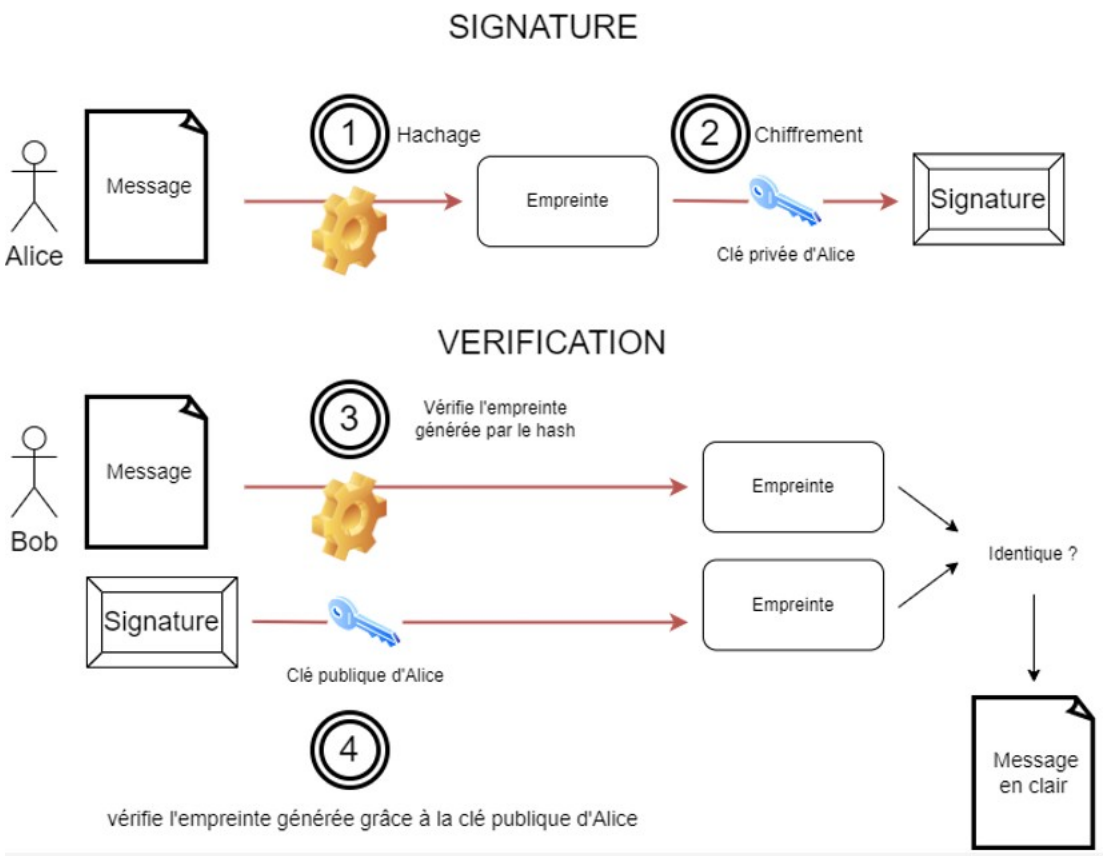
Le scellement consiste à calculer une empreinte (**sceller**) à partir d'un message et d'une clé privée pour authentifier l'origine des données et en vérifier l'intégrité.
Le scellement d'une empreinte génère un **sceau** ou code d'authentification de message (**MAC**). Il est réalisé au moyen d'une **fonction de hachage** appliquée au **message + clé privée**.



a

Concevez une signature numérique

La signature numérique est composée de **données ajoutées à un message** permettant au destinataire d'authentifier l'auteur du message électronique. Seul l'expéditeur est capable de générer la signature. La signature électronique s'appuie sur l'utilisation d'une fonction de hachage combinée à de la cryptographie asymétrique. La signature numérique revêt en France la même valeur légale qu'une signature papier.



a

Élaborez un certificat électronique

Le certificat électronique est un bloc composé de :

- La clé publique d'une paire de clés asymétriques.
- Des informations identifiant le porteur de cette paire de clés telles que son nom, son adresse IP, son adresse de messagerie, son URL, etc.
- L'identité de l’autorité de certification comme Verisign, EnTrust.net, CyberTrust ou CertPlus.
- La signature numérique des données ci-dessus.

Il est géré tout au long de son cycle de vie par l’autorité de Certification.

Pour aller plus loin : bcrypt et passport

Étudiez un module de hachage comme bcrypt ou un module d’authentification comme passport.

Je retiens : la cryptographie

Les mécanismes cryptographiques sont : le chiffrement (symétrique, asymétrique, hybride), le scellement (hachage/seau), la signature (scellement, chiffrement) et le certificat électronique (asymétrique/signature).

Identifiez le protocole IPSec

L'**IPSec** (Internet Protocol Security) est un ensemble de protocoles qui utilisent des algorithmes pour transporter des données sur un réseau IP de façon sécurisée. C’est un cadre standard ouvert défini par L'**IETF** (Internet Engineering Task Force).
L'IPSec est également l'une des méthodes pour la création de **VPN** (Virtual Private Network ou réseau privé virtuel). Il permet la liaison entre deux systèmes informatiques en prenant appui sur un réseau existant.
L'IPSec est lié aux protocoles **IPv4** et **IPv6**. Il permet d'authentifier et de chiffrer des données, afin d’assurer la confidentialité et l’intégrité des flux des données.

IPV4

IPV4 signifie "Internet Protocol Version 4". C’est la première version mise en œuvre au sein de l'ARPANET en 1983.

IPV6

IPV6 signifie "Internet Protocol Version 6" qui change complètement le fonctionnement de la communication en réseau.

- Augmentation de l’espace d’adressage de 32 à 128 bits, ce qui prévient le risque de ruptures d’adresses.
- Communication claire et sans ambiguïté. IPV6 applique formellement l’idée des IP.

Découvrez les autres protocoles

Le protocole SSL/TLS

TLS (Transport Layer Security) est le successeur du protocole **SSL** (Secure Socket Layer). C’est une nouvelle version qui utilise le chiffrement pour protéger le transfert des données, en s’appuyant sur un système de **certificat SSL**.
Ce certificat est installé sur le serveur et si un navigateur se connecte, le certificat SSL déclenche le protocole SSL (ou TLS) qui va chiffrer les informations envoyées entre le serveur et le navigateur (ou entre les serveurs). Il permet l’expression du protocole **HTTPS**.

Le protocole HTTPS

HTTPS (Hyper Text Transfer Protocol Secure) est une extension sécurisée du protocole HTTP, le « S » est ajouté pour « **Secured** ». Les données échangées entre le navigateur et le serveur hébergeant le site ou le service sont chiffrées et ne peuvent être lues ou modifiées. HTTPS n'est pas un protocole distinct de HTTP. Il utilise le chiffrement SSL/TLS sur HTTP pour sécuriser les communications.

Le protocole SSH : Secure Shell

SSH ou **Secure Shell** est une méthode de connexion à distance sécurisée d'une machine à une autre. Il offre plusieurs options d'authentification forte et protège la sécurité et l'intégrité des

communications par un chiffrement fort. Il propose une alternative sécurisée aux protocoles de connexion non protégés comme **TELNET** ou **RLOGIN** ou encore aux méthodes de transfert de fichiers non sécurisées comme **FTP**.

Le protocole SET

SET (Secure Electronic Transaction) est un système qui a pour objectif de garantir la sécurité des transactions électroniques par carte de crédit. Plusieurs techniques de chiffrement et de hachage viennent sécuriser les paiements sur Internet. Le protocole SET a été soutenu par des entreprises comme Visa, Mastercard et Microsoft qui a fourni sa technologie **STT** et Netscape qui a fourni **SSL**.

Pour aller plus loin : recommandations de l'ANSSI

Lisez les recommandations de l'ANSSI relatives à TLS.

Je retiens : les principaux protocoles de sécurisation réseau

Les principaux protocoles de sécurisation réseau sont : IPSec (transport de la donnée sur un réseau IP), SSL/TLS (chiffrement pour le transport), HTTPS (extension sécurisée du protocole http), SSH (méthode de connexion à distance) et SET (méthode de transaction par carte de crédit).

Découvrez les attaques DOS et Brute Force

Selon une étude de la société américaine Akamai, 83 % du trafic web est généré par des APIs. La sécurité est donc un enjeu de sécurité crucial.
Petit tour d'horizon des principales failles de sécurité à connaître…

Denial of service

Une **attaque DoS** (Denial of Service), ou déni de service, est une attaque réseau qui a pour objectif d'empêcher l'utilisation des ressources d'un serveur en le surchargeant de requêtes. Il existe plusieurs catégories d'attaques :

- Attaque sur la bande passante ;
- Saturation par des requêtes sur des services ;
- Attaque SYN flooding ;
- Attaque ICMP flood ;
- Attaque pair-à-pair ;
- Attaque DoS permanente ;
- Attaque par saturation au niveau application.

Brute Force

C’est une attaque consistant à envoyer un flux continu de requêtes, pour tester toutes les combinaisons possibles d’un mot de passe par exemple. L’objectif est de s’emparer des accès légitimes à un compte. C’est une méthode d’attaque simple à réaliser et efficace. Elle est très utilisée.

Solution

Pour ces deux types d’attaque, la solution est d’implémenter des mécanismes de Rate Limiting. Le principe de ces mécanismes repose sur la limitation du nombre de requêtes qu’un client donné est autorisé à envoyer dans un laps de temps défini. Cette solution permet d’empêcher qu’un client tier ne puissent s’accaparrer toutes les ressources d’un serveur (voir CloudFlare).

Identifiez le Cross-site Scripting

Le **cross-site scripting** (faille **XSS**) est un type de faille de sécurité des sites web permettant d'injecter du contenu, généralement du code JavaScript.
L’injection SQL permet quant à elle d’injecter du code SQL depuis des formulaires sur des sites qui sont mal protégés contre ce genre d’attaques.

Injection de code

Si l’attaquant connaît le langage de programmation utilisé pour l’API, il est en mesure d’injecter du code en utilisant les champs de saisie de texte pour forcer le serveur web à exécuter certaines instructions.

Injection SQL

L’attaquant injecte des commandes SQL afin d’interagir avec la base de données pour voler, modifier ou supprimer des données. Ces failles peuvent conduire à une compromission totale du serveur.
L’injection SQL est possible via les champs d’entrée de texte des formulaires. Si les données sont mal protégées, il est alors possible d’ajouter du code SQL qui sera exécuté au même moment ou le langage serveur envoie une requête SQL au serveur de base de données.

Solution

La solution passe par la validation des entrées utilisateurs (**Never trust user input**) et pour les injections SQL, l’utilisation de requêtes préparées (**prepared statements**).

Déterminez les failles d'authentification

On utilise souvent un token JWT pour authentifier les utilisateurs. Une mauvaise configuration côté serveur devient souvent une vulnérabilité. Il en va de même pour OAuth 2.0 & OpenID, très utilisés pour l'authentification.

Le Man in the middle

Une attaque **Man In the Middle** est un type d'attaque dans lequel une personne malveillante s'insère dans une communication ou un transfert de données entre un client et un serveur.

Solution

Il faut chiffrer les données avec le protocole TLS.

IDOR

Les références directes d'objet non sécurisées (insecure direct object references ou IDOR) sont des failles courantes qui permettent aux attaquants de contourner les autorisations et d'accéder directement aux ressources.

Solution

Il faut vérifier systématiquement les droits des utilisateurs.

Pour aller plus loin : 10 principaux risques relevé par l'OWASP

Parcourez ce document qui traite des 10 principaux risques relevé par l'OWASP.

Je retiens : Les principales failles de sécurité

Les principales failles sont le déni de service, le vol des accès, l'injection de code ou de SQL, le transfert de données et la mauvaise configuration côté serveur de l'authentification et des droits utilisateurs.

Identifiez les conditions du test

Il existe trois types de conditions de test :

- **La boîte noire** : le pentesteur n'a aucune information sur le réseau qu'il attaque et ne connaît aucun nom d'utilisateur ni mot de passe.
- **La boîte grise** : le pentesteur connaît un nombre limité d'informations.

- **La boîte blanche** : le pentesteur connaît un grand nombre d’informations.

Définition du périmètre du test

Selon que vous effectuez un test interne ou externe, il faudra définir le périmètre du test. Cela peut-être un serveur (application, serveur interne), une partie du réseau, un réseau local ou encore une ou plusieurs IP publiques.

Aspects légaux

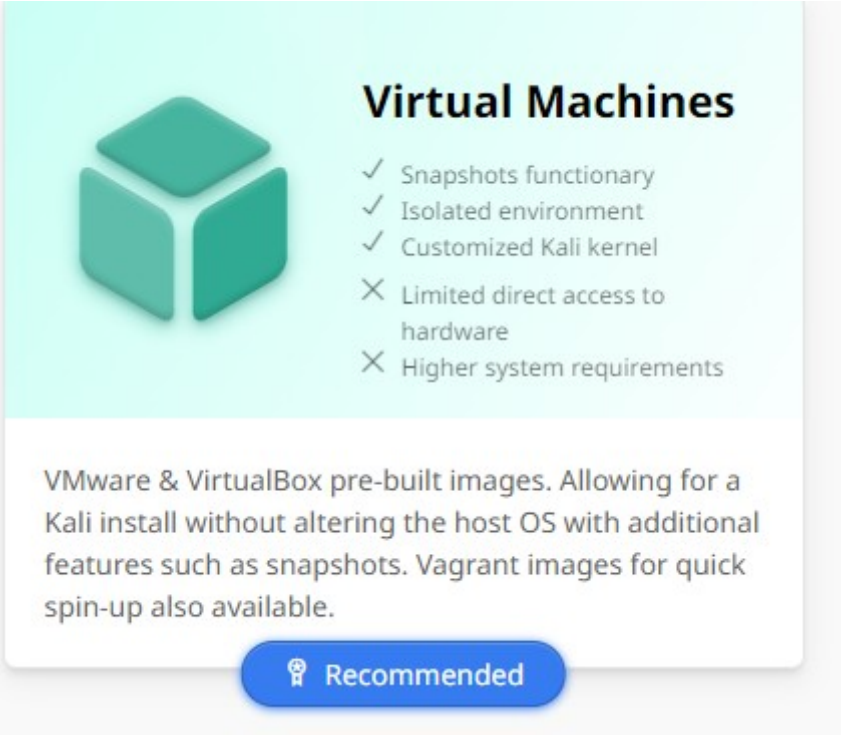
Un accord écrit et signé est indispensable. Un test d’intrusion sans autorisation est pénalement condamnable (Article 462-2 du Code pénal). Il est nécessaire d’établir ce mandat dans lequel sont précisées la date, l'heure du test et l'adresse IP publique du pentester en cas de test d'intrusion externe.

Outils

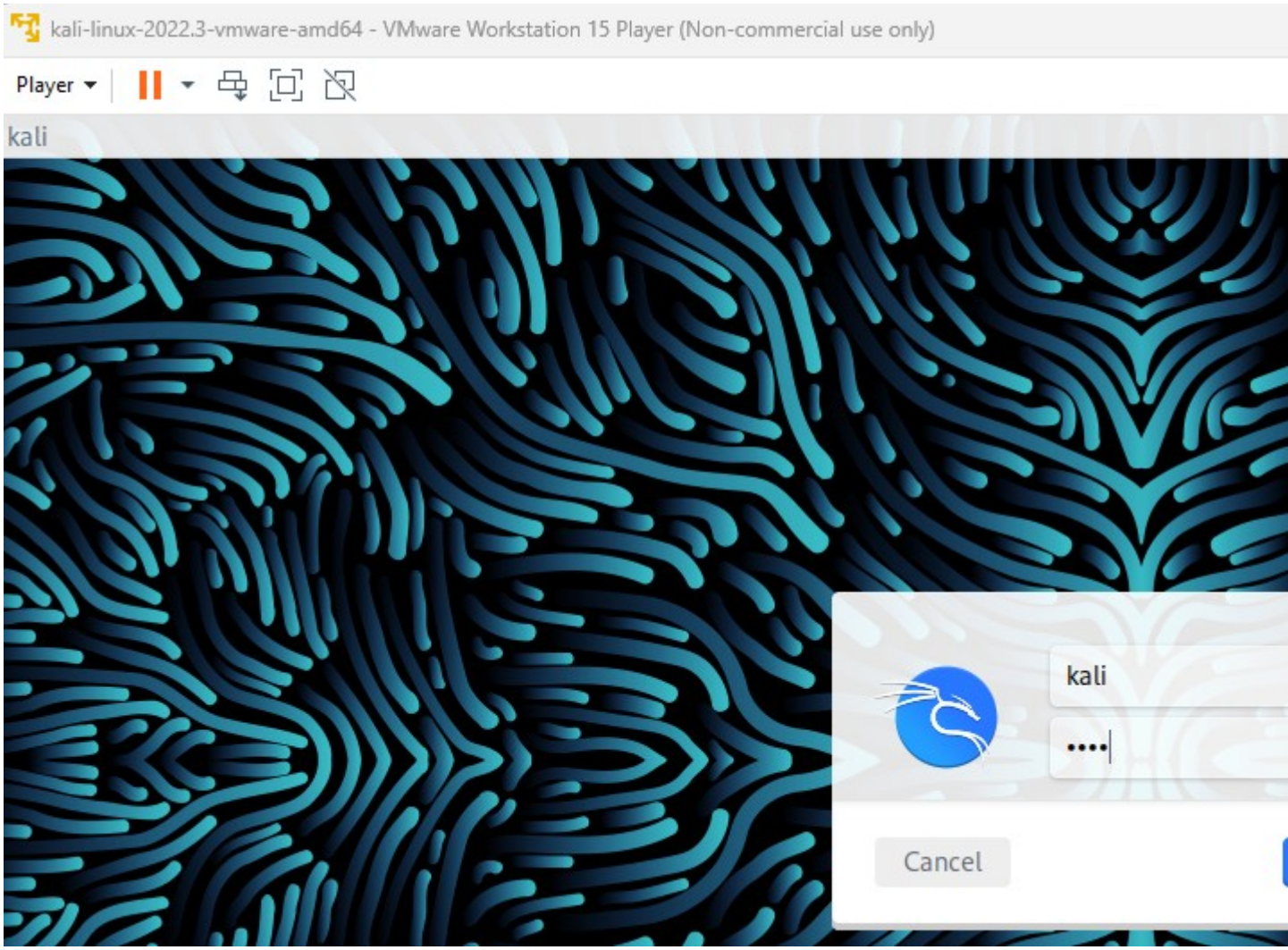
Les outils les plus utilisés sont ceux de la distribution **Kali Linux**. Des centaines de programmes sont disponibles (on les trouve sous Windows également) dont :

- **Nmap** : c’est un scanner conçu pour détecter les ports ouverts, identifier les services hébergés et obtenir des informations sur le système d'exploitation.
- **Wireshark** : il permet l’analyse de réseaux informatiques.
- **Metasploit** : c’est un programme qui trouve et exploite des informations sur les vulnérabilités.
- **Burp Suite** : tests d’intrusion des applications web.

Installez **Kali-linux** avec une machine virtuelle pré-installée par exemple :



Ci-dessous, sur VMWare Workstation Player avec comme identifiant **kali** et mot de passe **kali.**



Si vous le souhaitez, vous pouvez également créer une nouvelle machine virtuelle de A à Z à l'aide du logiciel gratuit VirtualBox.

Identifiez les ports ouverts

Avant le scan, vous pouvez identifier tous les ports ouverts sur une machine distante sur le réseau avec la commande **nmap.**

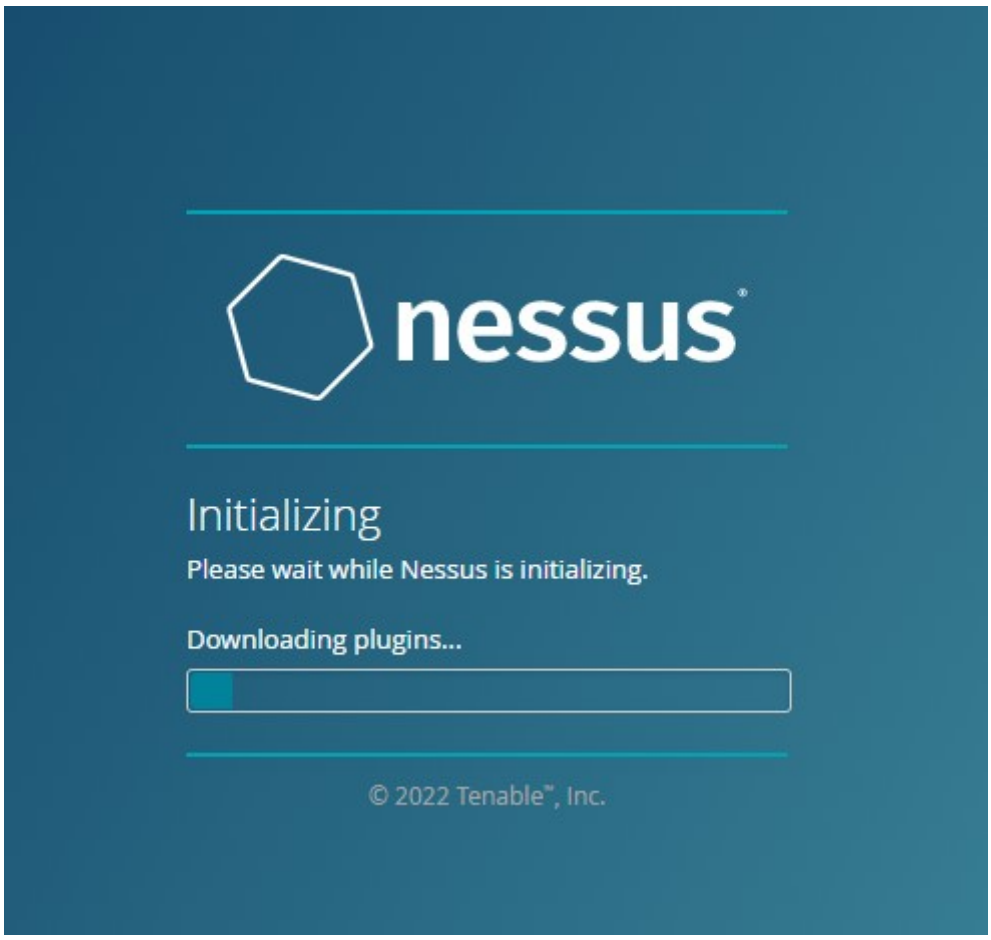
```
┌──(kali㉿kali)-[~]
└─$ nmap -p 3000 192.168.0.0/24
Starting Nmap 7.92 ( https://nmap.org ) at 2022-11-15 03:09 EST
Nmap scan report for 192.168.0.1
Host is up (0.0062s latency).

PORT     STATE    SERVICE
3000/tcp filtered ppp

Nmap scan report for 192.168. ■■■ ■ ■
Host is up (0.037s latency).

PORT     STATE    SERVICE
3000/tcp filtered ppp

Nmap scan report for 192.168.0.255
Host is up (0.0050s latency).

PORT     STATE  SERVICE
3000/tcp closed ppp

Nmap done: 256 IP addresses (3 hosts up) scanned in 3.51 seconds
```

Ici par exemple, on teste les ports 3000 de toutes les machines du réseau et on identifie une machine sur lequel il est utilisé. C'est le serveur node de l'application en train de fonctionner.

Scannez les vulnérabilités

Pour le scan de vulnérabilité, il existe un outil performant et facile à prendre en main : **Nessus.**
Il est payant mais peut-être utilisé gratuitement de manière personnelle. Téléchargez la version adaptée à votre distribution de Kali Linux.
Pour le démarrer, dans le terminal tapez :
/bin/systemctl start nessusd.service

Sous Windows, un exécutable est disponible.



Une fois Nessus installé, vous pouvez accéder à l'adresse https://kali:8834/#/ ou https://localhost:8834/ **sous Windows** et sélectionner **Nessus Essentials.**

nessus Essentials

Scans Settings

FOLDERS

My Scans

All Scans

Trash

RESOURCES

Policies

Plugin Rules

Terrascan

Scan Templates

‹ Back to Scans

Scanner

DISCOVERY

Host Discovery
A simple scan to discover live hosts and open ports.

VULNERABILITIES

Basic Network Scan
A full system scan suitable for any host.

Advanced Scan
Configure a scan without using any recommendations.

Advanc
Configure without

Intel AMT Security Bypass
Remote and local checks for CVE-2017-5689.

Spectre and Meltdown
Remote and local checks for CVE-2017-5753, CVE-2017-5715, and CVE-2017-5754

Wanna
Remote

PrintNightmare
Local checks to detect the PrintNightmare Vulnerability in Windows Print Spooler.

Active Directory Starter Scan
Look for misconfigurations in Active Directory.

Detection

ContiLeaks
Detection of vulnerabilities revealed in the ContiLeaks chats.

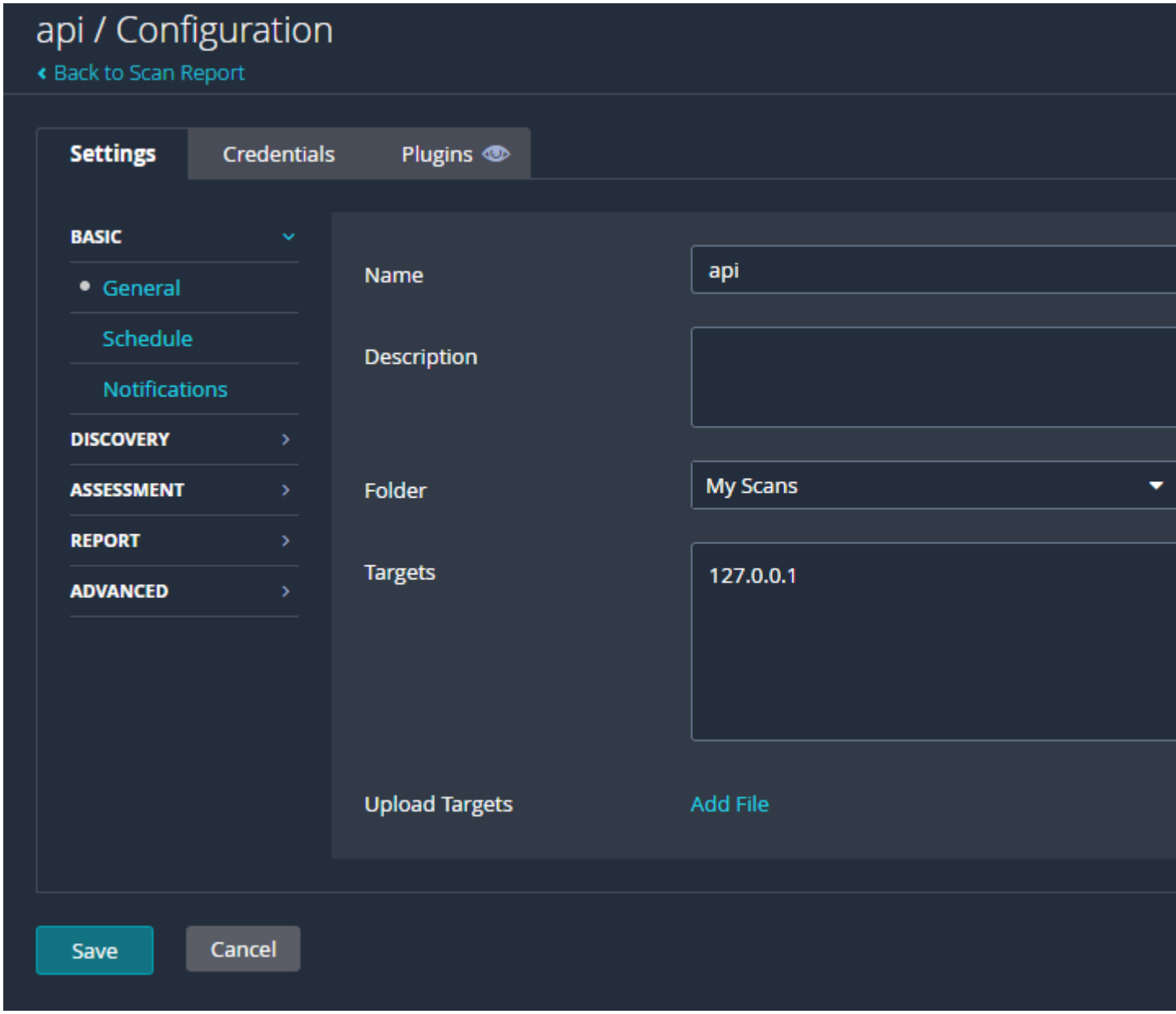
Ransomware Ecosystem
Vulnerabilities used by ransomware groups and affiliates.

Tenable News

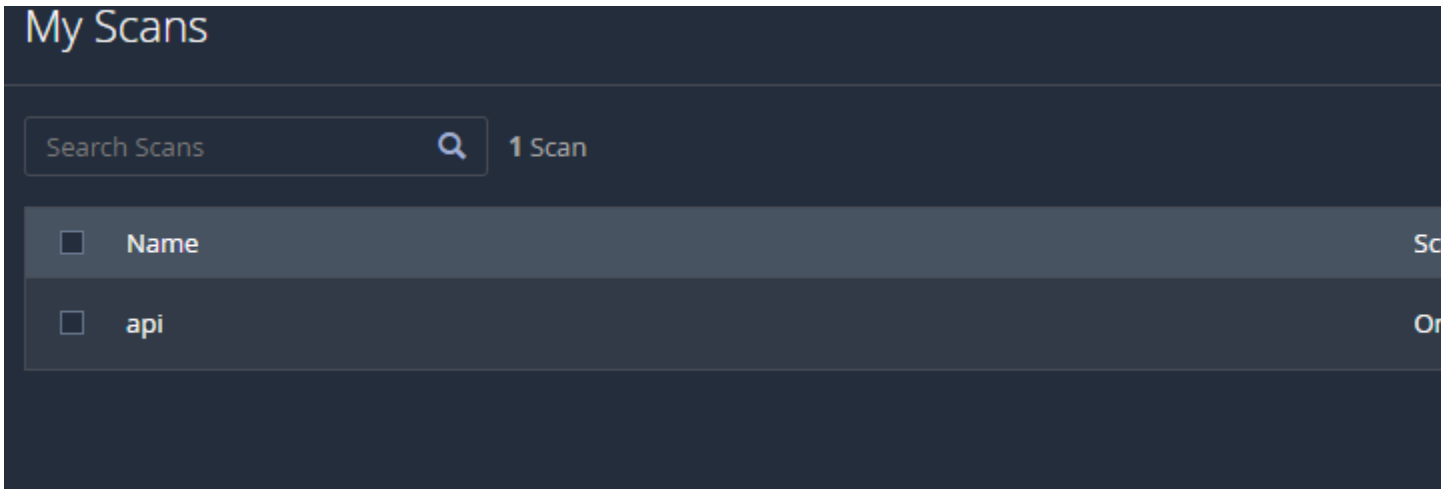
Cybersecurity Snapshot: Salary Trends, Ransomware ...

Read More

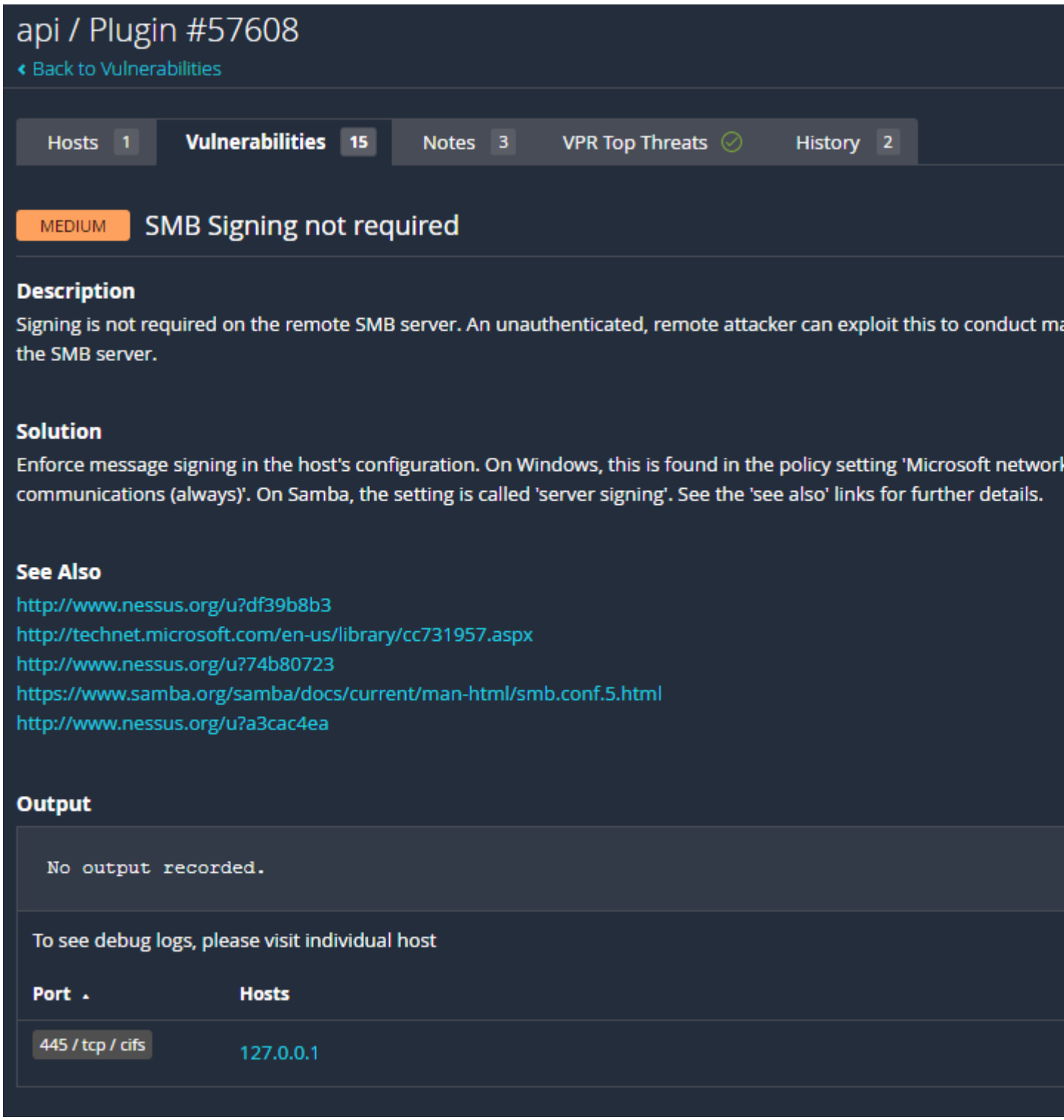
Après avoir créé un compte, initialisez votre espace de travail Nessus. Sélectionnez **Basic Network Scan** pour créer un scan basique. Essayez de scanner localhost par exemple.



Renseignez la partie credentials avec les identifiants de connexion de votre système d'exploitation. Une fois créé, vous pouvez lancer le scan avec la flèche qui se trouve à droite :



Une fois l’analyse achevée, vous avez accès à un rapport contenant les vulnérabilités découvertes et une description de chacune permettant de proposer des correctifs par la suite.



Une fois les vulnérabilités découvertes, il s'agit de tenter de les exploiter et de fournir un rapport avec des propositions correctives.

Pour aller plus loin : scan des applications web

Parcourez la documentation Nessus à propos du scan des applications web.

Je retiens : le test d'intrusion

Un test d'intrusion est un processus encadré par la loi qui se découpe en plusieurs étapes : recherche des ports actifs avec la commande nmap suivie de l'IP du réseau, scan de vulnérabilités avec un test basique sur Nessus, rapport et actions correctives.

Découvrez Mocha

Mocha est l'un des **lanceurs de test (frameworks de test)** les plus utilisés parmi les nombreux existants.
Voici une liste non-exhaustive de ses fonctionnalités :

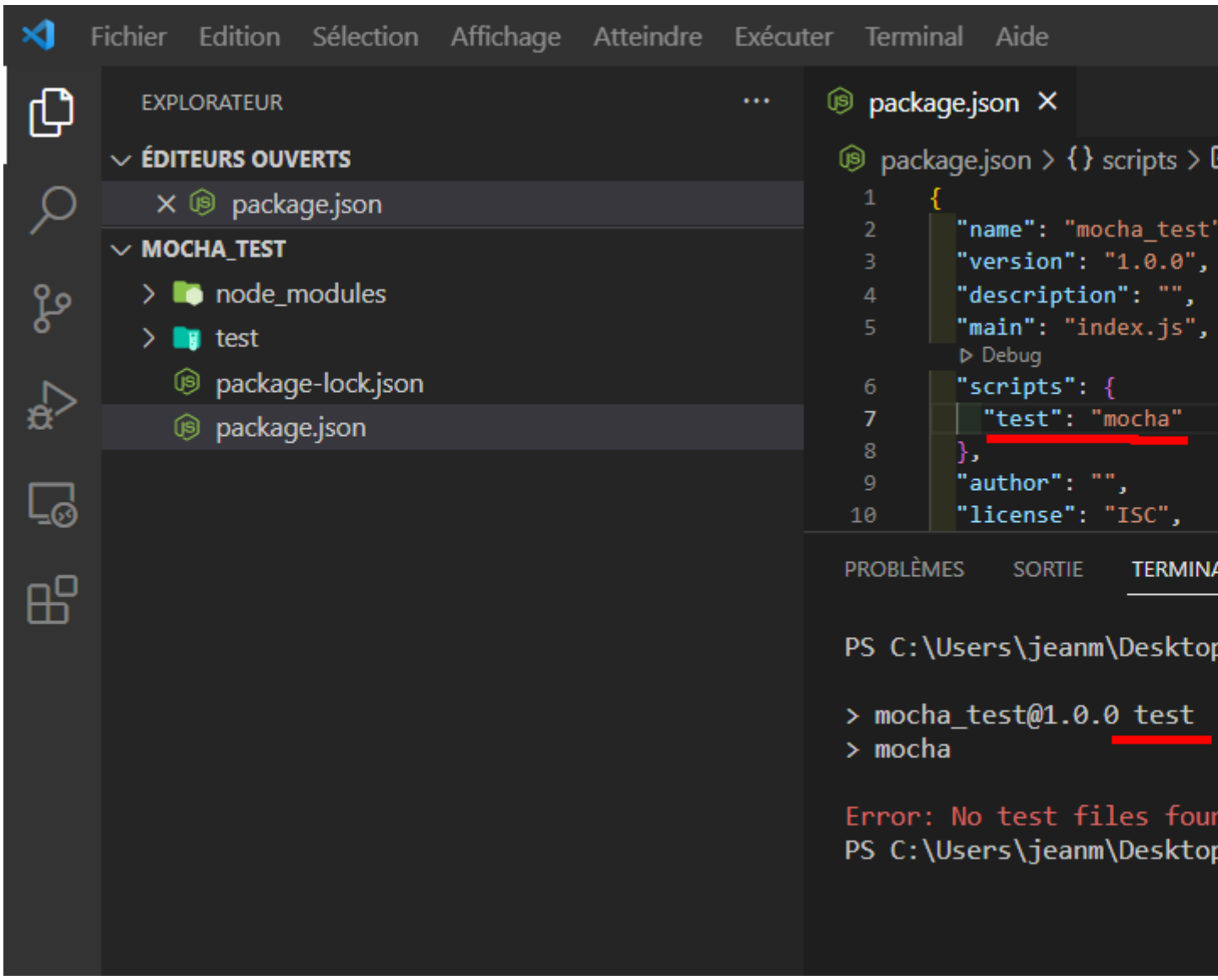
- Exécution des tests avec Node ;
- Exécution des tests dans un navigateur ;
- Support des promesses ;
- Génération de rapports sur la couverture des tests ;
- Facilité de génération des cas de tests ;
- Vérification des tests lents.

Installation :
Pour installer Mocha de manière globale sur votre système :
npm install -g mocha

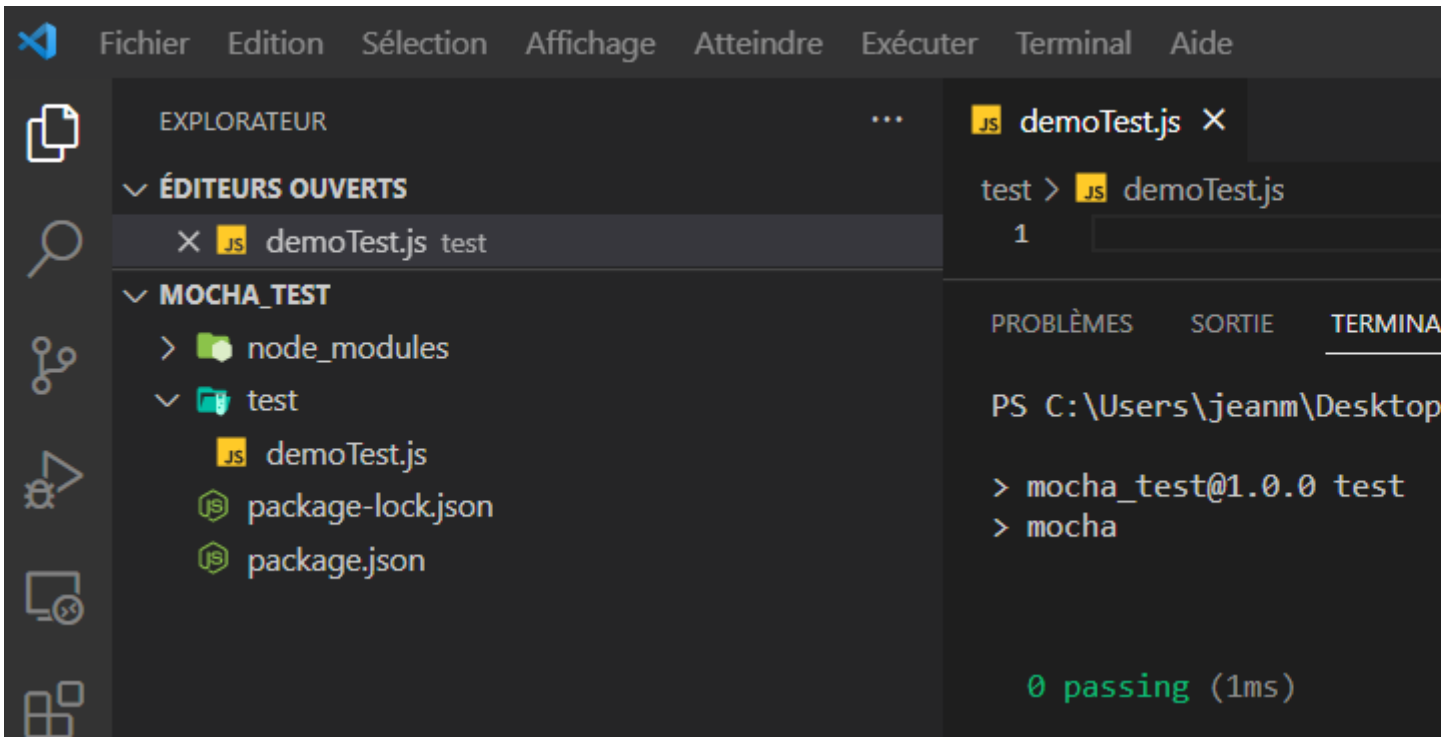
Vous pouvez vous passer de l'option **-g** et l'installer localement dans les dépendances de développement :
npm install mocha - - save-dev

Identifiez le répertoire test

Créez un répertoire projet **mocha_test** et initialisez-le avec **npm init**. À l'intérieur, créez un répertoire **test**. C'est dans ce répertoire que Mocha va, par défaut, chercher les fichiers JavaScript de tests à exécuter.
Modifiez l'index **test** dans la partie **scripts** du **package.json** en y plaçant la commande **mocha**. Et pour lancer mocha, utilisez la commande **npm test** :



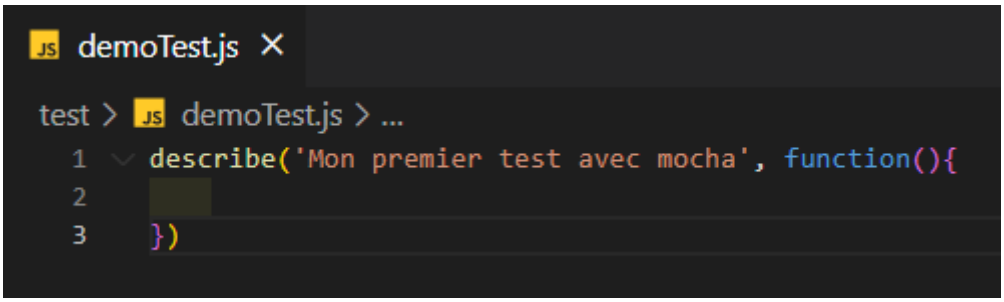
Créez dans **test** un fichier **demoTest.js**. Refaites un **npm test**.



Il a bien été trouvé un fichier de test et il est signalé qu'aucun test n'a été effectué. Cela fonctionne. Découvrez les différentes **manières d'écrire un test** en vous rendant dans la section **interfaces** de la documentation. Pour votre premier test, utilisez **l'interface BDD** qui est plus naturelle et facile à comprendre. Vous pourrez essayer d'autres interfaces si l'une d'entre elles vous inspire davantage.

Réalisez votre premier test Mocha

Dans **demoTest.js**, la première chose à faire est de **donner un nom à son test** en utilisant la méthode **describe()**. Elle prend deux arguments, le nom du test et la fonction callback appelée au lancement du test.



Votre test doit être découpé par étapes. Chaque étape va tester une fonctionnalité de votre application. Pour écrire l'une de ces étapes, utilisez la méthode **it().** Elle prend en arguments, un nom pour le test et une fonction de callback pour le lancer.

```
demoTest.js ×
test > demoTest.js > ...
1  describe('Mon premier test avec mocha', function(){
2      it('nature du test', function(){
3
4      })
5  })
```

Pour écrire le test à proprement parler, utilisez la librairie Node.Js **assert**. Elle permet d'exprimer une **assertion** (**affirmation**), le **résultat qui est attendu** et peut être accompagnée d'un message qui s'affichera en cas d'échec du test en troisième argument.

```
demoTest.js ×
test > demoTest.js > ...
1   // importation de la librairie assert
2   const assert = require('assert')
3
4   describe('Mon premier test avec mocha', function(){
5       it('Egalité', function(){
6           let n = 5
7           // Test utilisant assert.equal pour tester
8           // l'égalité de deux expressions
9           // Ici si n multiplié par deux est bien égal à 10
10          assert.equal(n*2, 10, 'La multiplication n\'a pas fonctionné')
11      })
12  })
```

Lancez le test avec **npm test**, vous devriez recevoir, en réponse dans la console, le nom du test avec un petit signe check vert pour signaler que le test est réussi et le nombre de tests effectués soit 1 dans notre cas (voir ci-dessous)

```
PS C:\Users\jeanm\Desktop\mocha_test> npm test

> mocha_test@1.0.0 test
> mocha



  Mon premier test avec mocha
    ✓ Egalité


  1 passing (4ms)
```

Changez la valeur attendue de l’assertion et le test ne sera pas un succès (voir ci-dessous).

demoTest.js

test > demoTest.js > ...

```
describe('Mon premier test avec mocha', function(){
    it('Egalité', function(){
        let n = 5
        // Test utilisant assert.equal pour tester
        // l'égalité de deux expressions
        // Ici si n multiplié par deux est bien égal à 10
        assert.equal(n*2, 11, 'La multiplication n\'a pas fonctionné')
    })
})
```

PROBLÈMES SORTIE TERMINAL

```
> mocha_test@1.0.0 test
> mocha



  Mon premier test avec mocha
    1) Egalité


  0 passing (10ms)
  1 failing

  1) Mon premier test avec mocha
       Egalité:

      AssertionError [ERR_ASSERTION]: La multiplication n'a pas fonctionné
      + expected - actual

      -10
      +11

      at Context.<anonymous> (test\demoTest.js:10:16)
      at process.processImmediate (node:internal/timers:471:21)
```

a

Testez un module

Créez un répertoire **services** et, à l'intérieur, un fichier **format.js** qui sera chargé de remplacer les espaces d'une chaîne de caractères par des underscores.

```
format.js ×
services > format.js > <unknown>
1  module.exports = {
2      formatName: function(name){
3          let formatName = name.split(' ').join('_')
4          return formatName
5      }
6  }
```

Testez ce module en modifiant demoTest.js (en anglais pour écrire un vrai test avec les bonnes pratiques).

```
demoTest.js ×
test > demoTest.js > ...
1   // import du module à tester
2   const format = require('../services/format')
3   const assert = require('assert')
4
5   describe('My second test using mocha', function(){
6       it('should return a string with an underscore instead of space character', fun
7           let name = "John Doe"
8           assert.equal(format.formatName(name), 'John_Doe', 'split of space characte
9       })
10  })
11
```

Testez :

```
PS C:\Users\jeanm\Desktop\mocha_test> npm test

> mocha_test@1.0.0 test
> mocha



  My second test using mocha
    ✓ should return a string with an underscore instead of space character


  1 passing (4ms)
```

a

Pour aller plus loin : la librairie Chaï

Le module Node.js pour les assertions. Utilisez la librairie Chaï comme moteur d’assertion.

Je retiens : Mocha

Mocha est un lanceur de test.
Il possède une méthode describe() pour créer des groupes de tests et it() pour lancer un de ces tests.
La librairie assert permet d’écrire les assertions dans le lanceur de test it().

Effectuez des tests manuels

Test d’acceptation / Acceptance test

Les tests d'acceptation vérifient si votre système répond aux exigences de logique métier. Ils simulent le comportement des utilisateurs.

Test de résilience / Resilience test

Ce test simule une activité réelle et des pannes d’environnement réel. Il vérifie que l’application continue à fonctionner. Il est proche du test de robustesse. Les pannes peuvent se provoquer à l’aide des chaos monkey.

Effectuez des tests automatisés

Test unitaire / Unit-test

Les tests unitaires sont de bas niveau et proche de votre application. Ils consistent à tester les méthodes des classes, les fonctions et les modules utilisés par votre application.

Test d’intégration / Integration test

Les tests d'intégration ont pour objectif de vérifier que les différents modules utilisés par votre application interagissent bien.

Test fonctionnel / Functional test

Les tests fonctionnels se contentent de vérifier la sortie d’un processus sans se soucier des états intermédiaires du système lors de l'exécution de ce processus.

Test de bout en bout / End-to-end test

Les tests de bout en bout sont réalisés dans un environnement applicatif dont le but est de reproduire le comportement d'un utilisateur afin de vérifier que les flux utilisateurs fonctionnent.

Test de charge

Il s’agit de simuler un nombre d'utilisateurs virtuels prédéfinis, afin de valider l'application pour une charge attendue d'utilisateurs.

Test de performance / Performance test

Les tests de performance permettent d’évaluer les performances de votre application à une certaine charge d’utilisation. Ils permettent de déterminer sa fiabilité et sa vitesse. Un test de performance peut observer le temps de réponse de requêtes mettant en jeu une grande quantité de données par exemple.

Test de dégradation des transactions / Degradation test of transactions

Ce test simule une activité transactionnelle parmi tous les scénarios du périmètre des tests, de manière à déterminer quelle charge le système est capable de supporter pour chaque scénario. Il permet d’isoler éventuellement les transactions qui dégradent le plus l’application.

Test de stress / Stress test

Ce test simule l'activité maximale attendue et teste tous les scénarios en heure de pointe de l'application.

Test de fiabilité / Reliability test

Ce test simule une charge importante d'utilisateurs sur une longue durée, pour voir si l’application est capable de supporter une activité intense sur une longue période.

Test de montée en charge / Scalability test

Ce test simule un nombre d'utilisateurs sans cesse croissant de manière à déterminer quelle charge limite le système est capable de supporter.

Test aux limites / Boundary test

Ce test simule une activité bien supérieure à l'activité normale. Il est proche du test de capacité mais il ne considère pas seulement l'augmentation d'un nombre d'utilisateurs simultanés.

Test de fumée / Smoke test

Le smoke test est simple et vérifie le fonctionnement de la base d'une application. Il s’assure que les fonctionnalités principales du système fonctionnent comme vous l’avez prévu. Il est utilisé après la création d'un build de l’application pour décider de tester plus profondément, ou après le déploiement pour s’assurer que l’application s’exécute correctement dans son nouvel environnement.

Pour aller plus loin : l'intégration continue

Il serait intéressant d’étudier comment effectuer des **tests en parallèle** de l’application. Vous pouvez également étudier la notion d’**intégration continue**.

Je retiens : les familles de tests

Les deux familles de tests sont les tests manuels et les tests automatisés.

Les différentes méthodes de test sont : Unit Test, Integration Test, Functional Test, End to end Test, Acceptance Test, Performance Test, Degradation test of transactions, Stress test, Reliability test, Scalability test, Boundary test, Resilience test et Smoke Test.